Trip Auditor Documentation

Generated by Doxygen 1.14.0

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Namespace Documentation

4.1 app Namespace Reference

Classes

- class AIReportGenerator
- class TripAuditorApp
- class UserManager

Variables

- app = TripAuditorApp()

4.1.1 Detailed Description

```
--------------------------------------------------------------------------------------------------------------
Start of Imports and Setup
--------------------------------------------------------------------------------------------------------------
```

4.1.2 Variable Documentation

4.1.2.1 app

```
app.app = TripAuditorApp()
```

Definition at line 627 of file app.py.

Chapter 5

Class Documentation

5.1 app.AIReportGenerator Class Reference

Static Public Member Functions

- generate (filtered_df)

5.1.1 Detailed Description

Definition at line 78 of file app.py.

5.1.2 Member Function Documentation

5.1.2.1 generate()

```
app.AIReportGenerator.generate (
              filtered_df)  [static]
```

Generate a summary AI report from the filtered DataFrame.

Definition at line 81 of file app.py.

```
00081     def generate(filtered_df):
00082         """Generate a summary AI report from the filtered DataFrame."""
00083         if filtered_df.empty:
00084             return "No data available for AI report."
00085         most_profitable_vehicle = filtered_df.groupby('Vehicle ID')['Net Profit'].sum().idxmax()
00086         top_routes = ", ".join(filtered_df['Route'].value_counts().head(2).index) if 'Route' in
      filtered_df.columns else "N/A"
00087         avg_profit_per_trip = round(filtered_df['Net Profit'].sum() / len(filtered_df), 2)
00088         rev = filtered_df['Freight Amount'].sum()
00089         exp = filtered_df['Total Trip Expense'].sum()
00090         profit = filtered_df['Net Profit'].sum()
00091         kms = filtered_df['Actual Distance (KM)'].sum()
00092         profit_pct = round((profit / rev * 100), 1) if rev else 0
00093         per_km = round(profit / kms, 2) if kms else 0
00094         return f"""
00095 AI Report Highlights:
00096
00097 Total Trips: {len(filtered_df)}
00098 On-going Trips: {filtered_df[filtered_df['Trip Status'] == 'Pending Closure'].shape[0]}
00099 Completed Trips: {filtered_df[filtered_df['Trip Status'] == 'Completed'].shape[0]}
```

```
00100 Profit Percentage: {profit_pct}%
00101
00102 Financials:
00103 - Revenue: Rs{round(rev / 1e6, 2)}M
00104 - Expense: Rs{round(exp / 1e6, 2)}M
00105 - Profit: Rs{round(profit / 1e6, 2)}M
00106 - KMs Travelled: {round(kms / 1e3, 1)}K
00107 - Cost per KM: Rs{per_km}
00108
00109 AI Insights:
00110 - Top Vehicle: {most_profitable_vehicle}
00111 - Average Profit per Trip: Rs{avg_profit_per_trip}
00112 - Top Routes: {top_routes}
00113 """
00114
00115 """
00116
      ------------------------------------------------------------------------------------------------------------------------

00117 Main Application Class
00118
      ------------------------------------------------------------------------------------------------------------------------

00119 [Summary] TripAuditorApp is the main Flask application for managing trip audits.
00120 [Details] It initializes the Flask app, loads datasets, manages user sessions, and defines all routes
      for the application.
00121
00122 """
00123
```

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

5.2 app.TripAuditorApp Class Reference

Public Member Functions

- __init__ (self)
- load_fleet_data (self)
- load_closure_data (self)
- register_routes (self)
- run (self)

Public Attributes

- app = Flask(__name__)
- str fleet_file = 'fleet_50_entries.xlsx'
- str closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
- df = self.load_fleet_data()
- closure_df = self.load_closure_data()
- vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
- str routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else [ ]
- user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))

5.2.1 Detailed Description

Definition at line 124 of file app.py.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 __init__()

```
app.TripAuditorApp.__init__ (
              self)
```

Definition at line 126 of file app.py.

```
00126     def __init__(self):
00127         # Initialize Flask app
00128         self.app = Flask(__name__)
00129         self.app.config['UPLOAD_FOLDER'] = 'uploads'
00130         os.makedirs(self.app.config['UPLOAD_FOLDER'], exist_ok=True)
00131         self.app.secret_key = 'supersecret'
00132 
00133         # Load datasets
00134         self.fleet_file = 'fleet_50_entries.xlsx'
00135         self.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00136         self.df = self.load_fleet_data()
00137         self.closure_df = self.load_closure_data()
00138 
00139         # Prepare vehicle and route lists
00140         self.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
00141         self.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else []
00142 
00143         # User management
00144         self.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
00145 
00146         # Register all routes
00147         self.register_routes()
00148 
```

5.2.3 Member Function Documentation

5.2.3.1 load_closure_data()

```
app.TripAuditorApp.load_closure_data (
              self)
```

Load and preprocess the closure Excel data.

Definition at line 159 of file app.py.

```
00159     def load_closure_data(self):
00160         """Load and preprocess the closure Excel data."""
00161         df = pd.read_excel(self.closure_file)
00162         df.columns = df.columns.str.strip()
00163         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00164         df['Day'] = df['Trip Date'].dt.day
00165         return df
00166 
```

5.2.3.2 load_fleet_data()

```
app.TripAuditorApp.load_fleet_data (
              self)
```

Load and preprocess the fleet Excel data.

Definition at line 150 of file app.py.

```
00150     def load_fleet_data(self):
00151         """Load and preprocess the fleet Excel data."""
00152         df = pd.read_excel(self.fleet_file)
00153         df.columns = df.columns.str.strip()
00154         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00155         df['Day'] = df['Trip Date'].dt.day
00156         return df
00157 
```

5.2.3.3 register_routes()

```
app.TripAuditorApp.register_routes (
              self)
```

Define all Flask routes for the app.

Definition at line 168 of file app.py.

```
00168     def register_routes(self):
00169         """Define all Flask routes for the app."""
00170 
00171         @self.app.route('/')
00172         def home():
00173             # Redirect to signup page
00174             return redirect(url_for('signup'))
00175 
00176         @self.app.route('/signup', methods=['GET', 'POST'])
00177         def signup():
00178             # Handle user signup
00179             if request.method == 'POST':
00180                 email = request.form['email']
00181                 if self.user_manager.email_exists(email):
00182                     return render_template('signup.html', error="Email already registered!")
00183                 self.user_manager.add_user(
00184                     name=request.form['fullname'],
00185                     email=email,
00186                     password=request.form['password']
00187                 )
00188                 return redirect(url_for('login'))
00189             return render_template('signup.html')
00190 
00191         @self.app.route('/login', methods=['GET', 'POST'])
00192         def login():
00193             # Handle user login
00194             if request.method == 'POST':
00195                 user = self.user_manager.find_user(request.form['email'])
00196                 if user and check_password_hash(user['password'], request.form['password']):
00197                     session['user'] = user
00198                     return redirect(url_for('dashboard'))
00199                 return render_template('login.html', error="Invalid credentials.")
00200             return render_template('login.html')
00201 
00202         @self.app.route('/dashboard')
00203         def dashboard():
00204             # Main dashboard with filters and summary
00205             if 'user' not in session:
00206                 return redirect(url_for('login'))
00207             vehicle = request.args.get('vehicle')
00208             route = request.args.get('route')
00209             filtered = self.df.copy()
00210             if vehicle:
00211                 filtered = filtered[filtered['Vehicle ID'] == vehicle]
00212             if route:
00213                 filtered = filtered[filtered['Route'] == route]
00214 
00215             total_trips = len(filtered)
00216             ongoing = filtered[filtered['Trip Status'] == 'Pending Closure'].shape[0]
00217             closed = filtered[filtered['Trip Status'] == 'Completed'].shape[0]
00218             flags = filtered[filtered['Trip Status'] == 'Under Audit'].shape[0]
00219             resolved = filtered[(filtered['Trip Status'] == 'Under Audit') & (filtered['POD Status']
      == 'Yes')].shape[0]
00220 
00221             rev = filtered['Freight Amount'].sum()
00222             exp = filtered['Total Trip Expense'].sum()
00223             profit = filtered['Net Profit'].sum()
00224             kms = filtered['Actual Distance (KM)'].sum()
00225 
00226             rev_m = round(rev / 1e6, 2)
00227             exp_m = round(exp / 1e6, 2)
00228             profit_m = round(profit / 1e6, 2)
00229             kms_k = round(kms / 1e3, 1)
00230             per_km = round(profit / kms, 2) if kms else 0
00231             profit_pct = round((profit / rev) * 100, 1) if rev else 0
00232 
00233             daily = filtered.groupby('Day')['Trip ID'].count().reindex(range(1, 32),
      fill_value=0).tolist()
00234             audited = filtered[filtered['Trip Status'] == 'Under Audit'].groupby('Day')['Trip
      ID'].count().reindex(range(1, 32), fill_value=0).tolist()
00235             audit_pct = [round(a / b * 100, 1) if b else 0 for a, b in zip(audited, daily)]
00236 
```

```
00237             bar_labels = ['Revenue', 'Expense', 'Profit']
00238             bar_values = [float(rev_m), float(exp_m), float(profit_m)]
00239             ai_report = AIReportGenerator.generate(filtered)
00240 
00241             return render_template('dashboard.html',
00242                 total_trips=total_trips, ongoing=ongoing, closed=closed,
00243                 flags=flags, resolved=resolved, rev_m=rev_m, exp_m=exp_m,
00244                 profit_m=profit_m, kms_k=kms_k, per_km=per_km, profit_pct=profit_pct,
00245                 ai_report=ai_report, vehicles=self.vehicles, routes=self.routes,
00246                 selected_vehicle=vehicle, selected_route=route,
00247                 daily=daily, audited=audited, audit_pct=audit_pct,
00248                 bar_labels=bar_labels, bar_values=bar_values)
00249 
00250         @self.app.route('/trip-generator', methods=['GET', 'POST'])
00251         def trip_generator():
00252             import sqlite3
00253             import os
00254             import fitz  # PyMuPDF
00255 
00256             UPLOAD_FOLDER = 'uploads'
00257             os.makedirs(UPLOAD_FOLDER, exist_ok=True)
00258 
00259             def init_db():
00260                 conn = sqlite3.connect('trips.db')
00261                 c = conn.cursor()
00262                 c.execute('''
00263                     CREATE TABLE IF NOT EXISTS trips (
00264                         trip_id TEXT,
00265                         trip_date TEXT,
00266                         vehicle_id TEXT,
00267                         driver_id TEXT,
00268                         planned_distance REAL,
00269                         advance_given REAL,
00270                         origin TEXT,
00271                         destination TEXT,
00272                         vehicle_type TEXT,
00273                         flags TEXT DEFAULT '',
00274                         total_freight REAL DEFAULT 0.0
00275                     )
00276                 ''')
00277                 conn.commit()
00278                 conn.close()
00279 
00280             def parse_pdf(filepath):
00281                 doc = fitz.open(filepath)
00282                 text = ""
00283                 for page in doc:
00284                     text += page.get_text()
00285 
00286                 result = {}
00287                 fields = [
00288                     'trip_id', 'trip_date', 'vehicle_id', 'driver_id', 'planned_distance',
00289                     'advance_given', 'origin', 'destination', 'vehicle_type', 'flags', 'total_freight'
00290                 ]
00291                 for field in fields:
00292                     for line in text.splitlines():
00293                         if field.replace("_", " ").lower() in line.lower():
00294                             try:
00295                                 value = line.split(":")[1].strip()
00296                             except:
00297                                 value = ""
00298                             result[field] = value
00299                             break
00300                     else:
00301                         result[field] = ""
00302 
00303                 try:
00304                     result['total_freight'] = float(result.get('total_freight', 0) or 0)
00305                 except:
00306                     result['total_freight'] = 0.0
00307 
00308                 return result
00309 
00310             # Initialize DB once
00311             init_db()
00312 
00313             parsed_data = {}
00314 
00315             if request.method == 'POST':
00316                 if 'pdf_file' in request.files and request.files['pdf_file'].filename != '':
00317                     pdf_file = request.files['pdf_file']
00318                     if pdf_file.filename.endswith('.pdf'):
00319                         filepath = os.path.join(UPLOAD_FOLDER, pdf_file.filename)
00320                         pdf_file.save(filepath)
00321                         parsed_data = parse_pdf(filepath)
00322                 else:
00323                     fields = [
```

```
00324                         'trip_id', 'trip_date', 'vehicle_id', 'driver_id', 'planned_distance',
00325                         'advance_given', 'origin', 'destination', 'vehicle_type', 'flags',
    'total_freight'
00326                     ]
00327                     data = []
00328                     for f in fields:
00329                         val = request.form.get(f, '')
00330                         if f == 'total_freight':
00331                             try:
00332                                 val = float(val)
00333                             except:
00334                                 val = 0.0
00335                         data.append(val)
00336 
00337                     conn = sqlite3.connect('trips.db')
00338                     c = conn.cursor()
00339                     c.execute(f"INSERT INTO trips ({','.join(fields)}) VALUES
    (?,?,?,?,?,?,?,?,?,?)", data)
00340                     conn.commit()
00341                     conn.close()
00342                     return redirect(url_for('trip_generator'))
00343 
00344             # Fetch summary statistics
00345             conn = sqlite3.connect('trips.db')
00346             c = conn.cursor()
00347             c.execute("SELECT COUNT(*) FROM trips")
00348             trip_count = c.fetchone()[0]
00349 
00350             c.execute("SELECT COUNT(*) FROM trips WHERE flags IS NOT NULL AND flags != ''")
00351             total_flags = c.fetchone()[0]
00352 
00353             c.execute("SELECT SUM(total_freight) FROM trips")
00354             total_freight = c.fetchone()[0] or 0.0
00355             conn.close()
00356 
00357             return render_template(
00358                 'trip_generator.html',
00359                 parsed_data=parsed_data,
00360                 trip_count=trip_count,
00361                 total_flags=total_flags,
00362                 total_freight=total_freight
00363             )
00364 
00365         @self.app.route('/trip-closure')
00366         def trip_closure():
00367             import sqlite3
00368             import os
00369 
00370             # Ensure DB and folders exist
00371             os.makedirs(self.app.config['UPLOAD_FOLDER'], exist_ok=True)
00372 
00373             def init_db():
00374                 conn = sqlite3.connect('trips.db')
00375                 c = conn.cursor()
00376                 c.execute('''CREATE TABLE IF NOT EXISTS trips (
00377                     trip_id TEXT PRIMARY KEY,
00378                     trip_date TEXT,
00379                     vehicle_id TEXT,
00380                     driver_id TEXT,
00381                     planned_distance REAL,
00382                     advance_given REAL,
00383                     origin TEXT,
00384                     destination TEXT,
00385                     vehicle_type TEXT,
00386                     flags TEXT DEFAULT '',
00387                     total_freight REAL DEFAULT 0.0
00388                 )''')
00389                 c.execute('''CREATE TABLE IF NOT EXISTS trip_closure (
00390                     trip_id TEXT PRIMARY KEY,
00391                     actual_distance REAL,
00392                     actual_delivery_date TEXT,
00393                     trip_delay_reason TEXT,
00394                     fuel_quantity REAL,
00395                     fuel_rate REAL,
00396                     fuel_cost REAL,
00397                     toll_charges REAL,
00398                     food_expense REAL,
00399                     lodging_expense REAL,
00400                     miscellaneous_expense REAL,
00401                     maintenance_cost REAL,
00402                     loading_charges REAL,
00403                     unloading_charges REAL,
00404                     penalty_fine REAL,
00405                     total_trip_expense REAL,
00406                     freight_amount REAL,
00407                     incentives REAL,
00408                     net_profit REAL,
```

```
00409                     payment_mode TEXT,
00410                     pod_status TEXT,
00411                     trip_status TEXT
00412                 )"')
00413                 conn.commit()
00414                 conn.close()
00415
00416             init_db()
00417
00418             fields = [
00419                 ('actual_distance', 'Actual Distance (KM)', 'number'),
00420                 ('actual_delivery_date', 'Actual Delivery Date', 'date'),
00421                 ('trip_delay_reason', 'Trip Delay Reason', 'text'),
00422                 ('fuel_quantity', 'Fuel Quantity (L)', 'number'),
00423                 ('fuel_rate', 'Fuel Rate', 'number'),
00424                 ('fuel_cost', 'Fuel Cost', 'number'),
00425                 ('toll_charges', 'Toll Charges', 'number'),
00426                 ('food_expense', 'Food Expense', 'number'),
00427                 ('lodging_expense', 'Lodging Expense', 'number'),
00428                 ('miscellaneous_expense', 'Miscellaneous Expense', 'number'),
00429                 ('maintenance_cost', 'Maintenance Cost', 'number'),
00430                 ('loading_charges', 'Loading Charges', 'number'),
00431                 ('unloading_charges', 'Unloading Charges', 'number'),
00432                 ('penalty_fine', 'Penalty/Fine', 'number'),
00433                 ('total_trip_expense', 'Total Trip Expense', 'number'),
00434                 ('freight_amount', 'Freight Amount', 'number'),
00435                 ('incentives', 'Incentives', 'number'),
00436                 ('net_profit', 'Net Profit', 'number'),
00437                 ('payment_mode', 'Payment Mode', 'text'),
00438                 ('pod_status', 'POD Status', 'text'),
00439                 ('trip_status', 'Trip Status', 'text')
00440             ]
00441
00442             parsed_data = {}
00443
00444             if request.method == 'POST':
00445                 trip_id = request.form.get('trip_id', '').strip()
00446                 if not trip_id:
00447                     return "Trip ID is required", 400
00448
00449                 data = [trip_id]
00450                 for f, label, ftype in fields:
00451                     val = request.form.get(f, '')
00452                     if ftype == 'number':
00453                         try:
00454                             val = float(val) if val != '' else 0.0
00455                         except:
00456                             val = 0.0
00457                     data.append(val)
00458
00459                 conn = sqlite3.connect('trips.db')
00460                 c = conn.cursor()
00461                 placeholders = ','.join('?' * len(data))
00462                 c.execute(f"'
00463                     INSERT OR REPLACE INTO trip_closure (
00464                         trip_id, {','.join(f for f, _, _ in fields)}
00465                     ) VALUES ({placeholders})
00466                 "', data)
00467                 conn.commit()
00468                 conn.close()
00469                 return redirect(url_for('trip_closure'))
00470
00471             # GET method - fetch stats and closures
00472             conn = sqlite3.connect('trips.db')
00473             c = conn.cursor()
00474
00475             c.execute("SELECT COUNT(*) FROM trip_closure")
00476             total_closures = c.fetchone()[0]
00477
00478             c.execute("SELECT SUM(total_trip_expense), SUM(net_profit) FROM trip_closure")
00479             sums = c.fetchone()
00480             total_expense = sums[0] or 0.0
00481             total_profit = sums[1] or 0.0
00482
00483             c.execute("SELECT * FROM trip_closure ORDER BY trip_id DESC")
00484             closures = c.fetchall()
00485             conn.close()
00486
00487             return render_template(
00488                 'trip_closer.html',
00489                 parsed_data=parsed_data,
00490                 total_closures=total_closures,
00491                 total_expense=total_expense,
00492                 total_profit=total_profit,
00493                 closures=closures,
00494                 fields=fields
00495             )
```

```
00496
00497         @self.app.route('/trip-auditor')
00498         def trip_auditor():
00499             import pandas as pd
00500             from flask import request, render_template
00501
00502             EXCEL_FILE = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00503
00504             def load_data():
00505                 df = pd.read_excel(EXCEL_FILE)
00506                 df.columns = df.columns.str.strip().str.title()  # Clean column names
00507                 return df
00508
00509             df = load_data()
00510
00511             trip_id = request.args.get('trip_id')  # If trip_id is passed, show details
00512
00513             if trip_id:
00514                 # Audit detail view
00515                 if 'Trip Id' not in df.columns:
00516                     return "<h1 style='color:white'>No Trip Id column found in data.</h1>"
00517
00518                 trip = df[df['Trip Id'].astype(str) == str(trip_id)]
00519                 if trip.empty:
00520                     return f"<h1 style='color:white'>Trip ID {trip_id} not found.</h1>"
00521
00522                 return render_template('trip_audit_detail.html', trip=trip.iloc[0].to_dict())
00523
00524             # Dashboard summary view
00525             status_col = df['Status'] if 'Status' in df.columns else pd.Series(dtype=str)
00526             audited_col = df['Audited'] if 'Audited' in df.columns else pd.Series(dtype=str)
00527             flag_col = df['Flag'] if 'Flag' in df.columns else pd.Series(dtype=str)
00528             trip_id_col = df['Trip Id'] if 'Trip Id' in df.columns else pd.Series(dtype=str)
00529
00530             total_trips = len(df)
00531             opened = len(df[status_col.str.lower() == 'open']) if not status_col.empty else 0
00532             audited = len(df[audited_col.str.lower() == 'yes']) if not audited_col.empty else 0
00533             closed = len(df[status_col.str.lower() == 'closed']) if not status_col.empty else 0
00534             audit_closed = len(df[
00535                 (status_col.str.lower() == 'closed') & (audited_col.str.lower() == 'yes')
00536             ]) if not status_col.empty and not audited_col.empty else 0
00537             flags = len(df[flag_col.str.lower() == 'yes']) if not flag_col.empty else 0
00538
00539             trip_data = df[['Trip Id']].dropna().to_dict('records') if 'Trip Id' in df.columns else []
00540
00541             return render_template(
00542                 'trip_auditor.html',
00543                 total_trips=total_trips,
00544                 opened=opened,
00545                 audited=audited,
00546                 closed=closed,
00547                 audit_closed=audit_closed,
00548                 flags=flags,
00549                 trips=trip_data
00550             )
00551
00552
00553         @self.app.route('/trip-ongoing')
00554         def trip_ongoing():
00555             # Show ongoing trips
00556             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
      'Trip Status']]
00557             return render_template('table_page.html', title="Ongoing Trips",
      table=data.to_html(classes='text-white', index=False))
00558
00559         @self.app.route('/trip-stats')
00560         def trip_stats():
00561             # Show trip statistics by day
00562             days = list(range(1, 32))
00563             total = self.df.groupby('Day')['Trip ID'].count().reindex(days, fill_value=0).tolist()
00564             ongoing = self.df[self.df['Trip Status'] == 'Pending Closure'].groupby('Day')['Trip
      ID'].count().reindex(days, fill_value=0).tolist()
00565             closed = self.df[self.df['Trip Status'] == 'Completed'].groupby('Day')['Trip
      ID'].count().reindex(days, fill_value=0).tolist()
00566
00567             total_json = json.dumps(total)
00568             ongoing_json = json.dumps(ongoing)
00569             closed_json = json.dumps(closed)
00570
00571             total_sum = sum(total)
00572             ongoing_sum = sum(ongoing)
00573             closed_sum = sum(closed)
00574
00575             return render_template('trip_stats.html',
00576                 total_data=total_json, ongoing_data=ongoing_json, closed_data=closed_json,
00577                 total_sum=total_sum, ongoing_sum=ongoing_sum, closed_sum=closed_sum)
00578
```

```
00579         @self.app.route('/financial-dashboard')
00580         def financial_dashboard():
00581             # Show financial dashboard with recent 10 days data
00582             df_fin = self.closure_df.copy()
00583             recent_days = sorted(df_fin['Day'].dropna().unique())[-10:]
00584             day_labels = [f"Day {int(d)}" for d in recent_days]
00585             daily = df_fin[df_fin['Day'].isin(recent_days)]
00586             revenue_data = daily.groupby('Day')['Freight Amount'].sum().reindex(recent_days,
      fill_value=0).astype(int).tolist()
00587             expense_data = daily.groupby('Day')['Total Trip Expense'].sum().reindex(recent_days,
      fill_value=0).astype(int).tolist()
00588             profit_data = [r - e for r, e in zip(revenue_data, expense_data)]
00589             total_revenue = round(df_fin['Freight Amount'].sum() / 1e6, 2)
00590             total_profit = round(df_fin['Net Profit'].sum() / 1e6, 2)
00591             total_km = round(df_fin['Actual Distance (KM)'].sum() / 1e3, 1)
00592             per_km = round(df_fin['Net Profit'].sum() / df_fin['Actual Distance (KM)'].sum(), 2) if
      df_fin['Actual Distance (KM)'].sum() else 0
00593
00594             return render_template('financial_dashboard.html',
00595                 days=day_labels, revenue=revenue_data, expense=expense_data, profit=profit_data,
00596                 total_revenue=total_revenue, total_profit=total_profit, total_km=total_km,
      per_km=per_km)
00597
00598         @self.app.route('/logout')
00599         def logout():
00600             # Log out the current user
00601             session.pop('user', None)
00602             return redirect(url_for('login'))
00603
00604         @self.app.route('/download-summary')
00605         def download_summary():
00606             # Download AI report summary as a text file
00607             filtered = self.df
00608             report = AIReportGenerator.generate(filtered)
00609             with open("AI_Report_Summary.txt", 'w', encoding='utf-8') as f:
00610                 f.write(report)
00611             return send_file("AI_Report_Summary.txt", as_attachment=True)
00612
```

5.2.3.4 run()

```
app.TripAuditorApp.run (
              self)
```

Run the Flask app.

Definition at line 613 of file app.py.

```
00613     def run(self):
00614         """Run the Flask app."""
00615         self.app.run(host='0.0.0.0', port=7860, debug=True)
00616
00617 """
00618     ------------------------------------------------------------------------------------------------------------------------
00619 Entry Point
00620
      ------------------------------------------------------------------------------------------------------------------------
00621 This is the entry point for the application, where the TripAuditorApp is created and run.
00622
00623 """
00624
```

5.2.4 Member Data Documentation

5.2.4.1 app

```
app.TripAuditorApp.app = Flask(__name__)
```

Definition at line 128 of file app.py.

5.2.4.2 closure_df

```
app.TripAuditorApp.closure_df = self.load_closure_data()
```

Definition at line 137 of file app.py.

5.2.4.3 closure_file

```
str app.TripAuditorApp.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
```

Definition at line 135 of file app.py.

5.2.4.4 df

```
app.TripAuditorApp.df = self.load_fleet_data()
```

Definition at line 136 of file app.py.

5.2.4.5 fleet_file

```
str app.TripAuditorApp.fleet_file = 'fleet_50_entries.xlsx'
```

Definition at line 134 of file app.py.

5.2.4.6 routes

```
str app.TripAuditorApp.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.↩
df.columns else []
```

Definition at line 141 of file app.py.

5.2.4.7 user_manager

```
app.TripAuditorApp.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
```

Definition at line 144 of file app.py.

5.2.4.8 vehicles

```
app.TripAuditorApp.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
```

Definition at line 140 of file app.py.

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

5.3 app.UserManager Class Reference

Public Member Functions

- __init__ (self, user_file)
- load_users (self)
- save_users (self)
- add_user (self, name, email, password, role='Owner')
- find_user (self, email)
- email_exists (self, email)

Public Attributes

- user_file = user_file
- users = self.load_users()

5.3.1 Detailed Description

Definition at line 29 of file app.py.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 __init__()

```
app.UserManager.__init__ (
               self,
               user_file)
```

Definition at line 30 of file app.py.

```
00030     def __init__(self, user_file):
00031         self.user_file = user_file
00032         self.users = self.load_users()
00033
```

5.3.3 Member Function Documentation

5.3.3.1 add_user()

```
app.UserManager.add_user (
               self,
               name,
               email,
               password,
               role = 'Owner')
```

```
Add a new user and save.
```

Definition at line 49 of file app.py.

```
00049     def add_user(self, name, email, password, role='Owner'):
00050         """Add a new user and save."""
00051         self.users.append({
00052             'name': name,
00053             'email': email,
00054             'password': generate_password_hash(password),
00055             'role': role
00056         })
00057         self.save_users()
00058
```

5.3.3.2 email_exists()

```
app.UserManager.email_exists (
               self,
               email)
```

Check if an email is already registered.

Definition at line 65 of file app.py.

```
00065     def email_exists(self, email):
00066         """Check if an email is already registered."""
00067         return any(u['email'] == email for u in self.users)
00068 """
00069
00070     ------------------------------------------------------------------------------------------------------------------------
00071 AI Report Generator Class
00072
     ------------------------------------------------------------------------------------------------------------------------
00073 [Summary] AIReportGenerator generates a summary AI report from the filtered DataFrame.
00074 [Details] Defines a static method to create a report based on the filtered DataFrame, summarizing key
     metrics like total trips, profit percentage, and financials.
00075
00076 """
00077
```

5.3.3.3 find_user()

```
app.UserManager.find_user (
               self,
               email)
```

Find a user by email.

Definition at line 60 of file app.py.

```
00060     def find_user(self, email):
00061         """Find a user by email."""
00062         return next((u for u in self.users if u['email'] == email), None)
00063
```

5.3.3.4 load_users()

```
app.UserManager.load_users (
               self)
```

Load users from the JSON file, or return empty list if file doesn't exist.

Definition at line 35 of file app.py.

```
00035     def load_users(self):
00036         """Load users from the JSON file, or return empty list if file doesn't exist."""
00037         if os.path.exists(self.user_file):
00038             with open(self.user_file, 'r') as f:
00039                 return json.load(f)
00040         return []
00041
```

5.3.3.5 save_users()

```
app.UserManager.save_users (
              self)
```

Save the current users list to the JSON file.

Definition at line 43 of file app.py.

```
00043     def save_users(self):
00044         """Save the current users list to the JSON file."""
00045         with open(self.user_file, 'w') as f:
00046             json.dump(self.users, f)
00047
```

5.3.4 Member Data Documentation

5.3.4.1 user_file

```
app.UserManager.user_file = user_file
```

Definition at line 31 of file app.py.

5.3.4.2 users

```
app.UserManager.users = self.load_users()
```

Definition at line 32 of file app.py.

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

Chapter 6

File Documentation

6.1 D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py File Reference

Classes

- class app.UserManager
- class app.AIReportGenerator
- class app.TripAuditorApp

Namespaces

- namespace app

Variables

- app.app = TripAuditorApp()

6.2 D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

Go to the documentation of this file.

```
00001 """
00002 
      ------------------------------------------------------------------------------------------------------------------------
00003 Start of Imports and Setup
00004 
      ------------------------------------------------------------------------------------------------------------------------
00005 """
00006 
00007 from flask import Flask, render_template, request, redirect, url_for, session, send_file
00008 from werkzeug.security import generate_password_hash, check_password_hash
00009 import pandas as pd
00010 import json
00011 import os
00012 
00013 """
00014 
      ------------------------------------------------------------------------------------------------------------------------
00015 End of Imports and Setup
00016 
      ------------------------------------------------------------------------------------------------------------------------
```

```
00017 """
00018
00019
00020 """
00021
    ------------------------------------------------------------------------------------------------------------------------
00022  User Management Class
00023
    ------------------------------------------------------------------------------------------------------------------------
00024 [Summary] UserManager handles user registration, login, and management
00025 [Details] It loads users from a JSON file, allows adding new users, and checks for existing emails.
00026
00027 """
00028
00029 class UserManager:
00030     def __init__(self, user_file):
00031         self.user_file = user_file
00032         self.users = self.load_users()
00033
00034     # This method Load users from the JSON file or return an empty list if the file doesn't exist.
00035     def load_users(self):
00036         """Load users from the JSON file, or return empty list if file doesn't exist."""
00037         if os.path.exists(self.user_file):
00038             with open(self.user_file, 'r') as f:
00039                 return json.load(f)
00040         return []
00041
00042     # This method writes the users list to the specified JSON file.
00043     def save_users(self):
00044         """Save the current users list to the JSON file."""
00045         with open(self.user_file, 'w') as f:
00046             json.dump(self.users, f)
00047
00048     # This method adds a new user to the users list and saves it to the JSON file.
00049     def add_user(self, name, email, password, role='Owner'):
00050         """Add a new user and save."""
00051         self.users.append({
00052             'name': name,
00053             'email': email,
00054             'password': generate_password_hash(password),
00055             'role': role
00056         })
00057         self.save_users()
00058
00059     # This method finds a user by email in the users list, It returns the user dictionary if found, or
    None if not found.
00060     def find_user(self, email):
00061         """Find a user by email."""
00062         return next((u for u in self.users if u['email'] == email), None)
00063
00064     # This method checks if an email already exists in the users list, It returns True if the email is
    found, or False otherwise.
00065     def email_exists(self, email):
00066         """Check if an email is already registered."""
00067         return any(u['email'] == email for u in self.users)
00068
00069 """
00070
    ------------------------------------------------------------------------------------------------------------------------
00071 AI Report Generator Class
00072
    ------------------------------------------------------------------------------------------------------------------------
00073 [Summary] AIReportGenerator generates a summary AI report from the filtered DataFrame.
00074 [Details] Defines a static method to create a report based on the filtered DataFrame, summarizing key
    metrics like total trips, profit percentage, and financials.
00075
00076 """
00077
00078 class AIReportGenerator:
00079     # This method generates a summary AI report from the filtered DataFrame.
00080     @staticmethod
00081     def generate(filtered_df):
00082         """Generate a summary AI report from the filtered DataFrame."""
00083         if filtered_df.empty:
00084             return "No data available for AI report."
00085         most_profitable_vehicle = filtered_df.groupby('Vehicle ID')['Net Profit'].sum().idxmax()
00086         top_routes = ", ".join(filtered_df['Route'].value_counts().head(2).index) if 'Route' in
    filtered_df.columns else "N/A"
00087         avg_profit_per_trip = round(filtered_df['Net Profit'].sum() / len(filtered_df), 2)
00088         rev = filtered_df['Freight Amount'].sum()
00089         exp = filtered_df['Total Trip Expense'].sum()
00090         profit = filtered_df['Net Profit'].sum()
00091         kms = filtered_df['Actual Distance (KM)'].sum()
00092         profit_pct = round((profit / rev * 100), 1) if rev else 0
00093         per_km = round(profit / kms, 2) if kms else 0
00094         return f"""
00095 AI Report Highlights:
```

```
00096
00097 Total Trips: {len(filtered_df)}
00098 On-going Trips: {filtered_df[filtered_df['Trip Status'] == 'Pending Closure'].shape[0]}
00099 Completed Trips: {filtered_df[filtered_df['Trip Status'] == 'Completed'].shape[0]}
00100 Profit Percentage: {profit_pct}%
00101
00102 Financials:
00103 - Revenue: Rs{round(rev / 1e6, 2)}M
00104 - Expense: Rs{round(exp / 1e6, 2)}M
00105 - Profit: Rs{round(profit / 1e6, 2)}M
00106 - KMs Travelled: {round(kms / 1e3, 1)}K
00107 - Cost per KM: Rs{per_km}
00108
00109 AI Insights:
00110 - Top Vehicle: {most_profitable_vehicle}
00111 - Average Profit per Trip: Rs{avg_profit_per_trip}
00112 - Top Routes: {top_routes}
00113 """
00114
00115 """
00116
     ------------------------------------------------------------------------------------------------------------------------
00117 Main Application Class
00118
     ------------------------------------------------------------------------------------------------------------------------
00119 [Summary] TripAuditorApp is the main Flask application for managing trip audits.
00120 [Details] It initializes the Flask app, loads datasets, manages user sessions, and defines all routes
     for the application.
00121
00122 """
00123
00124 class TripAuditorApp:
00125     # This method initializes the Flask app, loads datasets, prepares vehicle and route lists, and
     sets up user management.
00126     def __init__(self):
00127         # Initialize Flask app
00128         self.app = Flask(__name__)
00129         self.app.config['UPLOAD_FOLDER'] = 'uploads'
00130         os.makedirs(self.app.config['UPLOAD_FOLDER'], exist_ok=True)
00131         self.app.secret_key = 'supersecret'
00132
00133         # Load datasets
00134         self.fleet_file = 'fleet_50_entries.xlsx'
00135         self.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00136         self.df = self.load_fleet_data()
00137         self.closure_df = self.load_closure_data()
00138
00139         # Prepare vehicle and route lists
00140         self.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
00141         self.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else []
00142
00143         # User management
00144         self.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
00145
00146         # Register all routes
00147         self.register_routes()
00148
00149     # This method loads and preprocesses the fleet Excel data, converting date columns and cleaning up
     column names.
00150     def load_fleet_data(self):
00151         """Load and preprocess the fleet Excel data."""
00152         df = pd.read_excel(self.fleet_file)
00153         df.columns = df.columns.str.strip()
00154         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00155         df['Day'] = df['Trip Date'].dt.day
00156         return df
00157
00158     # This method loads and preprocesses the closure Excel data, converting date columns and cleaning
     up column names.
00159     def load_closure_data(self):
00160         """Load and preprocess the closure Excel data."""
00161         df = pd.read_excel(self.closure_file)
00162         df.columns = df.columns.str.strip()
00163         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00164         df['Day'] = df['Trip Date'].dt.day
00165         return df
00166
00167     # This method registers all Flask routes for the application, defining the logic for each route.
00168     def register_routes(self):
00169         """Define all Flask routes for the app."""
00170
00171         @self.app.route('/')
00172         def home():
00173             # Redirect to signup page
00174             return redirect(url_for('signup'))
00175
00176         @self.app.route('/signup', methods=['GET', 'POST'])
```

```
00177         def signup():
00178             # Handle user signup
00179             if request.method == 'POST':
00180                 email = request.form['email']
00181                 if self.user_manager.email_exists(email):
00182                     return render_template('signup.html', error="Email already registered!")
00183                 self.user_manager.add_user(
00184                     name=request.form['fullname'],
00185                     email=email,
00186                     password=request.form['password']
00187                 )
00188                 return redirect(url_for('login'))
00189             return render_template('signup.html')
00190 
00191         @self.app.route('/login', methods=['GET', 'POST'])
00192         def login():
00193             # Handle user login
00194             if request.method == 'POST':
00195                 user = self.user_manager.find_user(request.form['email'])
00196                 if user and check_password_hash(user['password'], request.form['password']):
00197                     session['user'] = user
00198                     return redirect(url_for('dashboard'))
00199                 return render_template('login.html', error="Invalid credentials.")
00200             return render_template('login.html')
00201 
00202         @self.app.route('/dashboard')
00203         def dashboard():
00204             # Main dashboard with filters and summary
00205             if 'user' not in session:
00206                 return redirect(url_for('login'))
00207             vehicle = request.args.get('vehicle')
00208             route = request.args.get('route')
00209             filtered = self.df.copy()
00210             if vehicle:
00211                 filtered = filtered[filtered['Vehicle ID'] == vehicle]
00212             if route:
00213                 filtered = filtered[filtered['Route'] == route]
00214 
00215             total_trips = len(filtered)
00216             ongoing = filtered[filtered['Trip Status'] == 'Pending Closure'].shape[0]
00217             closed = filtered[filtered['Trip Status'] == 'Completed'].shape[0]
00218             flags = filtered[filtered['Trip Status'] == 'Under Audit'].shape[0]
00219             resolved = filtered[(filtered['Trip Status'] == 'Under Audit') & (filtered['POD Status']
      == 'Yes')].shape[0]
00220 
00221             rev = filtered['Freight Amount'].sum()
00222             exp = filtered['Total Trip Expense'].sum()
00223             profit = filtered['Net Profit'].sum()
00224             kms = filtered['Actual Distance (KM)'].sum()
00225 
00226             rev_m = round(rev / 1e6, 2)
00227             exp_m = round(exp / 1e6, 2)
00228             profit_m = round(profit / 1e6, 2)
00229             kms_k = round(kms / 1e3, 1)
00230             per_km = round(profit / kms, 2) if kms else 0
00231             profit_pct = round((profit / rev) * 100, 1) if rev else 0
00232 
00233             daily = filtered.groupby('Day')['Trip ID'].count().reindex(range(1, 32),
      fill_value=0).tolist()
00234             audited = filtered[filtered['Trip Status'] == 'Under Audit'].groupby('Day')['Trip
      ID'].count().reindex(range(1, 32), fill_value=0).tolist()
00235             audit_pct = [round(a / b * 100, 1) if b else 0 for a, b in zip(audited, daily)]
00236 
00237             bar_labels = ['Revenue', 'Expense', 'Profit']
00238             bar_values = [float(rev_m), float(exp_m), float(profit_m)]
00239             ai_report = AIReportGenerator.generate(filtered)
00240 
00241             return render_template('dashboard.html',
00242                 total_trips=total_trips, ongoing=ongoing, closed=closed,
00243                 flags=flags, resolved=resolved, rev_m=rev_m, exp_m=exp_m,
00244                 profit_m=profit_m, kms_k=kms_k, per_km=per_km, profit_pct=profit_pct,
00245                 ai_report=ai_report, vehicles=self.vehicles, routes=self.routes,
00246                 selected_vehicle=vehicle, selected_route=route,
00247                 daily=daily, audited=audited, audit_pct=audit_pct,
00248                 bar_labels=bar_labels, bar_values=bar_values)
00249 
00250         @self.app.route('/trip-generator', methods=['GET', 'POST'])
00251         def trip_generator():
00252             import sqlite3
00253             import os
00254             import fitz  # PyMuPDF
00255 
00256             UPLOAD_FOLDER = 'uploads'
00257             os.makedirs(UPLOAD_FOLDER, exist_ok=True)
00258 
00259             def init_db():
00260                 conn = sqlite3.connect('trips.db')
```

```
00261                 c = conn.cursor()
00262                 c.execute('''
00263                     CREATE TABLE IF NOT EXISTS trips (
00264                         trip_id TEXT,
00265                         trip_date TEXT,
00266                         vehicle_id TEXT,
00267                         driver_id TEXT,
00268                         planned_distance REAL,
00269                         advance_given REAL,
00270                         origin TEXT,
00271                         destination TEXT,
00272                         vehicle_type TEXT,
00273                         flags TEXT DEFAULT '',
00274                         total_freight REAL DEFAULT 0.0
00275                     )
00276                 ''')
00277                 conn.commit()
00278                 conn.close()
00279 
00280             def parse_pdf(filepath):
00281                 doc = fitz.open(filepath)
00282                 text = ""
00283                 for page in doc:
00284                     text += page.get_text()
00285 
00286                 result = {}
00287                 fields = [
00288                     'trip_id', 'trip_date', 'vehicle_id', 'driver_id', 'planned_distance',
00289                     'advance_given', 'origin', 'destination', 'vehicle_type', 'flags', 'total_freight'
00290                 ]
00291                 for field in fields:
00292                     for line in text.splitlines():
00293                         if field.replace("_", " ").lower() in line.lower():
00294                             try:
00295                                 value = line.split(":")[1].strip()
00296                             except:
00297                                 value = ""
00298                             result[field] = value
00299                             break
00300                     else:
00301                         result[field] = ""
00302 
00303                 try:
00304                     result['total_freight'] = float(result.get('total_freight', 0) or 0)
00305                 except:
00306                     result['total_freight'] = 0.0
00307 
00308                 return result
00309 
00310             # Initialize DB once
00311             init_db()
00312 
00313             parsed_data = {}
00314 
00315             if request.method == 'POST':
00316                 if 'pdf_file' in request.files and request.files['pdf_file'].filename != '':
00317                     pdf_file = request.files['pdf_file']
00318                     if pdf_file.filename.endswith('.pdf'):
00319                         filepath = os.path.join(UPLOAD_FOLDER, pdf_file.filename)
00320                         pdf_file.save(filepath)
00321                         parsed_data = parse_pdf(filepath)
00322                 else:
00323                     fields = [
00324                         'trip_id', 'trip_date', 'vehicle_id', 'driver_id', 'planned_distance',
00325                         'advance_given', 'origin', 'destination', 'vehicle_type', 'flags',
      'total_freight'
00326                     ]
00327                     data = []
00328                     for f in fields:
00329                         val = request.form.get(f, '')
00330                         if f == 'total_freight':
00331                             try:
00332                                 val = float(val)
00333                             except:
00334                                 val = 0.0
00335                         data.append(val)
00336 
00337                     conn = sqlite3.connect('trips.db')
00338                     c = conn.cursor()
00339                     c.execute(f"INSERT INTO trips ({','.join(fields)}) VALUES
      (?,?,?,?,?,?,?,?,?,?)", data)
00340                     conn.commit()
00341                     conn.close()
00342                     return redirect(url_for('trip_generator'))
00343 
00344             # Fetch summary statistics
00345             conn = sqlite3.connect('trips.db')
```

```
00346             c = conn.cursor()
00347             c.execute("SELECT COUNT(*) FROM trips")
00348             trip_count = c.fetchone()[0]
00349 
00350             c.execute("SELECT COUNT(*) FROM trips WHERE flags IS NOT NULL AND flags != ''")
00351             total_flags = c.fetchone()[0]
00352 
00353             c.execute("SELECT SUM(total_freight) FROM trips")
00354             total_freight = c.fetchone()[0] or 0.0
00355             conn.close()
00356 
00357             return render_template(
00358                 'trip_generator.html',
00359                 parsed_data=parsed_data,
00360                 trip_count=trip_count,
00361                 total_flags=total_flags,
00362                 total_freight=total_freight
00363             )
00364 
00365         @self.app.route('/trip-closure')
00366         def trip_closure():
00367             import sqlite3
00368             import os
00369 
00370             # Ensure DB and folders exist
00371             os.makedirs(self.app.config['UPLOAD_FOLDER'], exist_ok=True)
00372 
00373             def init_db():
00374                 conn = sqlite3.connect('trips.db')
00375                 c = conn.cursor()
00376                 c.execute('''CREATE TABLE IF NOT EXISTS trips (
00377                     trip_id TEXT PRIMARY KEY,
00378                     trip_date TEXT,
00379                     vehicle_id TEXT,
00380                     driver_id TEXT,
00381                     planned_distance REAL,
00382                     advance_given REAL,
00383                     origin TEXT,
00384                     destination TEXT,
00385                     vehicle_type TEXT,
00386                     flags TEXT DEFAULT '',
00387                     total_freight REAL DEFAULT 0.0
00388                 )''')
00389                 c.execute('''CREATE TABLE IF NOT EXISTS trip_closure (
00390                     trip_id TEXT PRIMARY KEY,
00391                     actual_distance REAL,
00392                     actual_delivery_date TEXT,
00393                     trip_delay_reason TEXT,
00394                     fuel_quantity REAL,
00395                     fuel_rate REAL,
00396                     fuel_cost REAL,
00397                     toll_charges REAL,
00398                     food_expense REAL,
00399                     lodging_expense REAL,
00400                     miscellaneous_expense REAL,
00401                     maintenance_cost REAL,
00402                     loading_charges REAL,
00403                     unloading_charges REAL,
00404                     penalty_fine REAL,
00405                     total_trip_expense REAL,
00406                     freight_amount REAL,
00407                     incentives REAL,
00408                     net_profit REAL,
00409                     payment_mode TEXT,
00410                     pod_status TEXT,
00411                     trip_status TEXT
00412                 )''')
00413                 conn.commit()
00414                 conn.close()
00415 
00416             init_db()
00417 
00418             fields = [
00419                 ('actual_distance', 'Actual Distance (KM)', 'number'),
00420                 ('actual_delivery_date', 'Actual Delivery Date', 'date'),
00421                 ('trip_delay_reason', 'Trip Delay Reason', 'text'),
00422                 ('fuel_quantity', 'Fuel Quantity (L)', 'number'),
00423                 ('fuel_rate', 'Fuel Rate', 'number'),
00424                 ('fuel_cost', 'Fuel Cost', 'number'),
00425                 ('toll_charges', 'Toll Charges', 'number'),
00426                 ('food_expense', 'Food Expense', 'number'),
00427                 ('lodging_expense', 'Lodging Expense', 'number'),
00428                 ('miscellaneous_expense', 'Miscellaneous Expense', 'number'),
00429                 ('maintenance_cost', 'Maintenance Cost', 'number'),
00430                 ('loading_charges', 'Loading Charges', 'number'),
00431                 ('unloading_charges', 'Unloading Charges', 'number'),
00432                 ('penalty_fine', 'Penalty/Fine', 'number'),
```

```
00433                 ('total_trip_expense', 'Total Trip Expense', 'number'),
00434                 ('freight_amount', 'Freight Amount', 'number'),
00435                 ('incentives', 'Incentives', 'number'),
00436                 ('net_profit', 'Net Profit', 'number'),
00437                 ('payment_mode', 'Payment Mode', 'text'),
00438                 ('pod_status', 'POD Status', 'text'),
00439                 ('trip_status', 'Trip Status', 'text')
00440             ]
00441
00442             parsed_data = {}
00443
00444             if request.method == 'POST':
00445                 trip_id = request.form.get('trip_id', '').strip()
00446                 if not trip_id:
00447                     return "Trip ID is required", 400
00448
00449                 data = [trip_id]
00450                 for f, label, ftype in fields:
00451                     val = request.form.get(f, '')
00452                     if ftype == 'number':
00453                         try:
00454                             val = float(val) if val != '' else 0.0
00455                         except:
00456                             val = 0.0
00457                     data.append(val)
00458
00459                 conn = sqlite3.connect('trips.db')
00460                 c = conn.cursor()
00461                 placeholders = ','.join('?' * len(data))
00462                 c.execute(f"""
00463                     INSERT OR REPLACE INTO trip_closure (
00464                         trip_id, {','.join(f for f, _, _ in fields)}
00465                     ) VALUES ({placeholders})
00466                 """, data)
00467                 conn.commit()
00468                 conn.close()
00469                 return redirect(url_for('trip_closure'))
00470
00471             # GET method - fetch stats and closures
00472             conn = sqlite3.connect('trips.db')
00473             c = conn.cursor()
00474
00475             c.execute("SELECT COUNT(*) FROM trip_closure")
00476             total_closures = c.fetchone()[0]
00477
00478             c.execute("SELECT SUM(total_trip_expense), SUM(net_profit) FROM trip_closure")
00479             sums = c.fetchone()
00480             total_expense = sums[0] or 0.0
00481             total_profit = sums[1] or 0.0
00482
00483             c.execute("SELECT * FROM trip_closure ORDER BY trip_id DESC")
00484             closures = c.fetchall()
00485             conn.close()
00486
00487             return render_template(
00488                 'trip_closer.html',
00489                 parsed_data=parsed_data,
00490                 total_closures=total_closures,
00491                 total_expense=total_expense,
00492                 total_profit=total_profit,
00493                 closures=closures,
00494                 fields=fields
00495             )
00496
00497         @self.app.route('/trip-auditor')
00498         def trip_auditor():
00499             import pandas as pd
00500             from flask import request, render_template
00501
00502             EXCEL_FILE = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00503
00504             def load_data():
00505                 df = pd.read_excel(EXCEL_FILE)
00506                 df.columns = df.columns.str.strip().str.title()  # Clean column names
00507                 return df
00508
00509             df = load_data()
00510
00511             trip_id = request.args.get('trip_id')  # If trip_id is passed, show details
00512
00513             if trip_id:
00514                 # Audit detail view
00515                 if 'Trip Id' not in df.columns:
00516                     return "<h1 style='color:white'>No Trip Id column found in data.</h1>"
00517
00518                 trip = df[df['Trip Id'].astype(str) == str(trip_id)]
00519                 if trip.empty:
```

```
00520                     return f"<h1 style='color:white'>Trip ID {trip_id} not found.</h1>"
00521
00522                 return render_template('trip_audit_detail.html', trip=trip.iloc[0].to_dict())
00523
00524             # Dashboard summary view
00525             status_col = df['Status'] if 'Status' in df.columns else pd.Series(dtype=str)
00526             audited_col = df['Audited'] if 'Audited' in df.columns else pd.Series(dtype=str)
00527             flag_col = df['Flag'] if 'Flag' in df.columns else pd.Series(dtype=str)
00528             trip_id_col = df['Trip Id'] if 'Trip Id' in df.columns else pd.Series(dtype=str)
00529
00530             total_trips = len(df)
00531             opened = len(df[status_col.str.lower() == 'open']) if not status_col.empty else 0
00532             audited = len(df[audited_col.str.lower() == 'yes']) if not audited_col.empty else 0
00533             closed = len(df[status_col.str.lower() == 'closed']) if not status_col.empty else 0
00534             audit_closed = len(df[
00535                 (status_col.str.lower() == 'closed') & (audited_col.str.lower() == 'yes')
00536             ]) if not status_col.empty and not audited_col.empty else 0
00537             flags = len(df[flag_col.str.lower() == 'yes']) if not flag_col.empty else 0
00538
00539             trip_data = df[['Trip Id']].dropna().to_dict('records') if 'Trip Id' in df.columns else []
00540
00541             return render_template(
00542                 'trip_auditor.html',
00543                 total_trips=total_trips,
00544                 opened=opened,
00545                 audited=audited,
00546                 closed=closed,
00547                 audit_closed=audit_closed,
00548                 flags=flags,
00549                 trips=trip_data
00550             )
00551
00552
00553         @self.app.route('/trip-ongoing')
00554         def trip_ongoing():
00555             # Show ongoing trips
00556             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
      'Trip Status']]
00557             return render_template('table_page.html', title="Ongoing Trips",
      table=data.to_html(classes='text-white', index=False))
00558
00559         @self.app.route('/trip-stats')
00560         def trip_stats():
00561             # Show trip statistics by day
00562             days = list(range(1, 32))
00563             total = self.df.groupby('Day')['Trip ID'].count().reindex(days, fill_value=0).tolist()
00564             ongoing = self.df[self.df['Trip Status'] == 'Pending Closure'].groupby('Day')['Trip
      ID'].count().reindex(days, fill_value=0).tolist()
00565             closed = self.df[self.df['Trip Status'] == 'Completed'].groupby('Day')['Trip
      ID'].count().reindex(days, fill_value=0).tolist()
00566
00567             total_json = json.dumps(total)
00568             ongoing_json = json.dumps(ongoing)
00569             closed_json = json.dumps(closed)
00570
00571             total_sum = sum(total)
00572             ongoing_sum = sum(ongoing)
00573             closed_sum = sum(closed)
00574
00575             return render_template('trip_stats.html',
00576                 total_data=total_json, ongoing_data=ongoing_json, closed_data=closed_json,
00577                 total_sum=total_sum, ongoing_sum=ongoing_sum, closed_sum=closed_sum)
00578
00579         @self.app.route('/financial-dashboard')
00580         def financial_dashboard():
00581             # Show financial dashboard with recent 10 days data
00582             df_fin = self.closure_df.copy()
00583             recent_days = sorted(df_fin['Day'].dropna().unique())[-10:]
00584             day_labels = [f"Day {int(d)}" for d in recent_days]
00585             daily = df_fin[df_fin['Day'].isin(recent_days)]
00586             revenue_data = daily.groupby('Day')['Freight Amount'].sum().reindex(recent_days,
      fill_value=0).astype(int).tolist()
00587             expense_data = daily.groupby('Day')['Total Trip Expense'].sum().reindex(recent_days,
      fill_value=0).astype(int).tolist()
00588             profit_data = [r - e for r, e in zip(revenue_data, expense_data)]
00589             total_revenue = round(df_fin['Freight Amount'].sum() / 1e6, 2)
00590             total_profit = round(df_fin['Net Profit'].sum() / 1e6, 2)
00591             total_km = round(df_fin['Actual Distance (KM)'].sum() / 1e3, 1)
00592             per_km = round(df_fin['Net Profit'].sum() / df_fin['Actual Distance (KM)'].sum(), 2) if
      df_fin['Actual Distance (KM)'].sum() else 0
00593
00594             return render_template('financial_dashboard.html',
00595                 days=day_labels, revenue=revenue_data, expense=expense_data, profit=profit_data,
00596                 total_revenue=total_revenue, total_profit=total_profit, total_km=total_km,
      per_km=per_km)
00597
00598         @self.app.route('/logout')
```

```
00599         def logout():
00600             # Log out the current user
00601             session.pop('user', None)
00602             return redirect(url_for('login'))
00603 
00604         @self.app.route('/download-summary')
00605         def download_summary():
00606             # Download AI report summary as a text file
00607             filtered = self.df
00608             report = AIReportGenerator.generate(filtered)
00609             with open("AI_Report_Summary.txt", 'w', encoding='utf-8') as f:
00610                 f.write(report)
00611             return send_file("AI_Report_Summary.txt", as_attachment=True)
00612 
00613     def run(self):
00614         """Run the Flask app."""
00615         self.app.run(host='0.0.0.0', port=7860, debug=True)
00616 
00617 """
00618 
    ------------------------------------------------------------------------------------------------------------------------
00619 Entry Point
00620 
    ------------------------------------------------------------------------------------------------------------------------
00621 This is the entry point for the application, where the TripAuditorApp is created and run.
00622 
00623 """
00624 
00625 if __name__ == "__main__":
00626     # Create and run the OOP-based TripAuditor app
00627     app = TripAuditorApp()
00628     app.run()
```